

Angular Forms

Angular Forms are a core feature of Angular that allows developers to handle user input, validation, and form submission efficiently. They provide tools to capture, validate, and process data entered into HTML forms, making implementing features like login forms, registrations, and complex multi-step forms easy.

Angular provides two ways to handle user input and form validation:

1. **Template-driven Forms:** Simple forms defined using Angular's template syntax, suitable for small-scale forms.
2. **Reactive Forms:** Forms with more control, built programmatically in TypeScript.

Template-driven Forms

Use Angular directives in templates to build forms.

Key Concepts in Template-driven Forms

1. **FormsModule:** Import FormsModule in your module to enable template-driven forms.
2. Best for simple forms with minimal logic.
3. **Two-way Data Binding:** Use `[(ngModel)]` to bind form inputs to model properties.
4. **Form Directives:**
 - **ngForm:** In Angular, ngForm is a directive provided by the FormsModule that represents the form's state and behavior. It is automatically applied to every `<form>` element when you use Angular template-driven forms.
 - **ngModel:** Binds input fields to the model.
5. **Validation:** Angular provides built-in validators like required, minlength, maxlength, pattern, and you can create custom ones.

Example: Simple Template-driven Form

app.module.ts

```
import { NgModule } from '@angular/core';  
import { BrowserModule } from '@angular/platform-browser';  
import { FormsModule } from '@angular/forms';  
import { AppComponent } from './app.component';
```

```
@NgModule({
```

```
declarations: [AppComponent],
imports: [BrowserModule, FormsModule], // Import FormsModule
providers: [],
bootstrap: [AppComponent],
})
export class AppModule {}
```

- **BrowserModule:** Essential module for any Angular web app. Provides directives and services for the app to run in the browser.
- **FormsModule:** Enables template-driven forms and provides directives like ngForm and ngModel.
- **AppComponent:** Declares the main application component.

app.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  user = {
    firstName: '',
    lastName: ''
  };

  onSubmit() {
    console.log('Form Submitted:', this.user);
    alert(`Hello, ${this.user.firstName} ${this.user.lastName}!`);
  }
}
```

Model Object:

- **user:** Represents the form data, with firstName and lastName as its properties.

- **Event Handler:**

- onSubmit():**

- Executes when the form is submitted.
 - Logs the user data to the console.
 - Displays an alert message with the entered firstName and lastName.

App.component.html

```
<div class="container">

  <h2>Simple Angular Form</h2>

  <form (ngSubmit)="onSubmit()" #simpleForm="ngForm">

    <!-- First Name -->

    <div>

      <label for="firstName">First Name:</label>

      <input type="text" id="firstName" name="firstName" [(ngModel)]="user.firstName" required/>

    </div>

    <!-- Last Name -->

    <div>

      <label for="lastName">Last Name:</label>

      <input type="text" id="lastName" name="lastName" [(ngModel)]="user.lastName" required />

    </div>

    <button type="submit" [disabled]="simpleForm.invalid">Submit</button>

  </form> </div>
```

? Form Declaration:

- **<form>:**
 - (ngSubmit)="onSubmit()": Calls the onSubmit() method when the form is submitted.
 - #simpleForm="ngForm": A **Local Reference Variable** in Angular is a variable declared in the template that gives you direct access to a DOM element, a directive, or a component instance within that template. These variables are defined using the # symbol followed by a name. Assigns a reference (simpleForm) to the form instance, exposing its properties and state (e.g., invalid).

? First Name Input:

- **id="firstName"**: Unique identifier for accessibility.
- **name="firstName"**: Required for Angular to track the control.
- **[(ngModel)]="user.firstName"**: Binds the input to the user.firstName property.
- **required**: Ensures the field cannot be empty.

🔗 **Last Name Input:**

- Similar to the First Name field but bound to user.lastName.

🔗 **Submit Button:**

- **[disabled]="simpleForm.invalid"**:
 - Disables the button if any field is invalid (e.g., empty firstName or lastName).

App.component.css

```
.container {
  width: 300px;
  margin: 20px auto;
  font-family: Arial, sans-serif;
}

label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}

input {
  width: 100%;
  padding: 5px;
  margin-bottom: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
}

button {
  padding: 10px 15px;
  background-color: #007bff;
```

```
color: white;

border: none;

border-radius: 4px;

cursor: pointer;
}

button:disabled {

background-color: #ccc;

}
```

Key Angular Features in the Code

1. Two-Way Data Binding:

- **[(ngModel)]:**
 - Syncs the user object properties (firstName and lastName) with the form inputs.

2. Form State:

- **simpleForm.invalid:**
 - Checks if the form is valid or invalid based on the required fields.

3. Event Binding:

- **(ngSubmit)="onSubmit()":**
 - Triggers the onSubmit() method when the form is submitted.

4. Validation:

- **required:**
 - Ensures fields are not left blank.

What Happens at Runtime?

1. The form is displayed with two input fields and a disabled **Submit** button.
2. The user enters values in the fields.
3. The **Submit** button becomes enabled when both fields are filled.
4. Upon clicking **Submit**:
 - The onSubmit() method logs the entered data.
 - Displays an alert with the user's full name.